

PRAKTISK INFORMASJON

DrOJ

INNHold

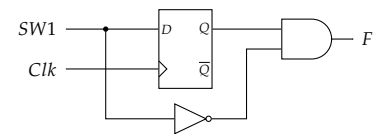
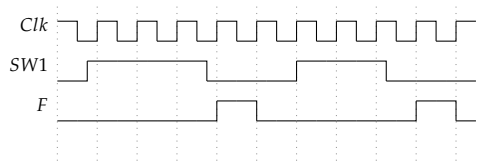
Egne blokker i Icestudio	2
Kontrollsignal	3
Prescaler: Tregere klokke	3
Blinking med D-vippe eller T-vippe	4
Trykknapper og prell (bouncing)	4
Skrive logikk med Verilog	5
Vektorer i Verilog og Icestudio	6
Binær-til-BCD for syvsegment	8
Vipper	9
Design med JK-vipper	10
Design med T-vipper	12
Skiftregistre	13
Skiftregistre i Verilog	14
Generator for tilfeldige tall	15

EGNE BLOKKER I ICESTUDIO

ICESTUDIO lar oss lage egne blokker. Oppgavene i praktisk del kan bli enklere å holde kontroll på om vi lager egne blokker. Første del av det praktiske grunnlaget av denne øvingen går derfor gjennom et eksempel fra forelesning og hvordan det kan lages egen blokk med egnet ikon.

I FORELESNING skal vi snakke om kretsen i figur 1 som detekterer at en trykknapp er trykket ned og slippes opp. Dette er å detektere negativ flanke på et signal siden bryterens binære verdi går fra 1 til 0. Dette er vist i figur 2:

- Legg merke til at trykknappens signal *SW1* ikke er i synk med klokken.
- Utgangen *F* til detektoren kan bare endre verdi når klokken har positiv flanke.
- Utgangen *F* blir høy når *SW1* går fra 1 til 0 rundt en positiv klokkeflanke.
- Utgangen går ned igjen ved neste positive klokkeflanke.



Figur 1: Deteksjon av negativ flanke.

Figur 2: Deteksjon av negativ flanke

LAGE EGNE ICESTUDIO-BLOKKER:

1. Lagres i egen fil.
2. Den kan hentes frem igjen fra menyen med `File > Add as block`
3. Innganger og utganger som lages fra `Basic > Input` og `Basic > Output` skal *ikke* ha navn og FPGA-pin skal ikke være huket av.
4. Dersom kretsen gjør bruk av klokke må det lages en egen klokkeinngang. For denne må det hukes av `Show clock`, se figur 3.

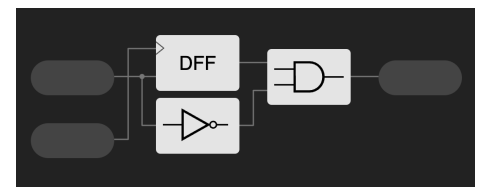
I ICESTUDIO kan kretsen i figur 1 nå lages som kretsen i figur 4.

IKONER for blokkene er ikke nødvendig, men det kan bli forvirrende med mange blanke blokker. Dersom vi ønsker å bruke egne ikoner til en blokk kan vi gå til menyen `Edit > Project Information`.

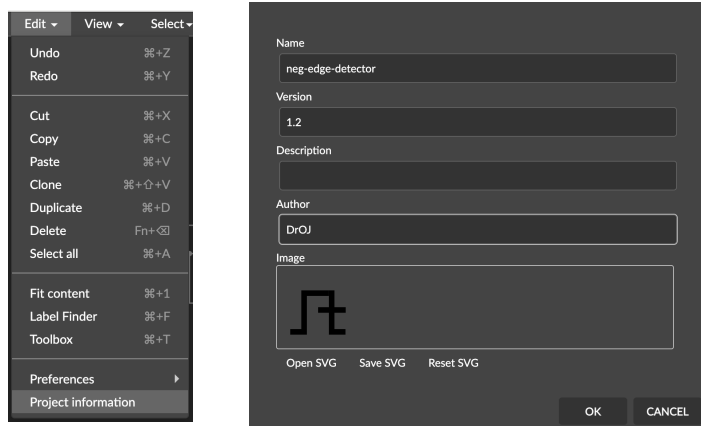
I DIALOGEN kan vi legge til en hel mengde informasjon om blokken samt legge ved en fil med ikonet. Disse må være `svg`-filer. Trykk `Open svg` og legg til en `svg`-fil. Bildene i figur 5 viser eksempel på prosjektinformasjon med ikon. Husk å lagre kretsen når du er ferdig.



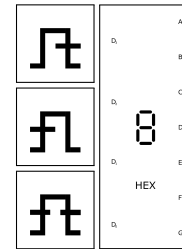
Figur 3: Definer klokkeinngang til en blokk.



Figur 4: Realisering av kretsen i figur 1.



Figur 5: Menyvalg og eksempel på prosjektinformasjon og ikon.



Figur 6: svg-filer tilgjengelig på Discord.

IKONENE i figur 6 er lagt til svg-filer på Discord, se #Digitalteknikk > filer.

KONTROLLSIGNAL

I EN SEKVENSIELL KRETS bør alle vipper styres med samme klokke. Men det er ikke alltid ønskelig at vipper skal endre tilstand en gang per klokkesignal. En enable-inngang kan brukes for å kontrollere om en vippe får endre tilstand. En annen måte er å lage et eget kontrollsignal *Ctrl*. Dette kan lages med for eksempel bryter. Til dette er for eksempel kretsen i figur 1 ideell, selv om det ofte er bedre å bruke positiv flanke på bryteren.

PRESCALER: TREGERE KLOKKE

DEN INTERNE KLOKKEN på et FPGA-kort er svært rask. For rask til at den kan brukes til å blinke en lysdiode for eksempel. lcestudio har derfor “Prescaler”-blokker som kan brukes til å lage en tregere klokke. En PrescalerN halverer klokkefrekvensen N ganger. Det vil si at den doubler perioden til klokken N ganger.

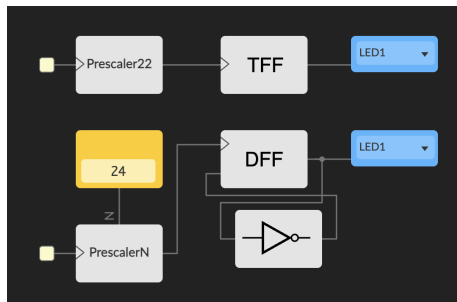
KLOKKEFREKVENSEN til den interne klokken på GoBoard er på 25 MHz som gir en klokkeperiode på 40 ns. Tydelig blinking med en LED bør ha en klokkeperiode på over 150 ms. Da kan vi løse for N og se at vi bør ha $N = 22$ eller større som vist i ligning (1).

lcestudio har en Prescaler22 innbygd og en generell Prescaler der vi kan legge inn en konstant som vi henter fra menyen Basic > Constant.

$$\begin{aligned}
 40 \text{ ns} \cdot 2^N &> 150 \text{ ms} \\
 2^N &> \frac{150 \text{ ms}}{40 \text{ ns}} \\
 2^N &> 3750000 \\
 \log_2 2^N &> \log_2 3750000 \\
 N &> 21,84 \\
 N &= 22 \quad (1)
 \end{aligned}$$

BLINKING MED D-VIPPE ELLER T-VIPPE

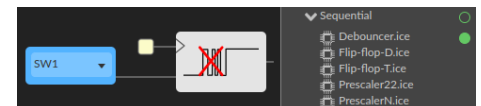
BLINKING i Icestudio kan realiseres med en D-vippe der vi setter inngangen til den inverterte av utgangen. En T-vippe gjør det samme; utgangen inverteres hver gang vippen trigges. To løsninger for å blinke lysdioder er vist i figur 7.



Figur 7: Blinking av LED i Icestudio.

TRYKKNAPPER OG PRELL (BOUNCING)

TRYKKNAPPER brukes ofte for å gi inngangssignaler når vi tester ut kretser i FPGA. Brytere kan oppleve *prell* (engelsk: bouncing). Det vil si at de skifter verdi mellom 0 og 1 flere ganger når de trykkes ned eller slipper opp. Å “renske” opp i et brytersignal kan gjøres med en “Debouncer”. En slik er implementert i Icestudio og vist brukt i figur 8.

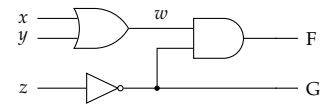


Figur 8: Debounce i Icestudio.

SKRIVE LOGIKK MED VERILOG

Med **VERILOG** kan vi skrive logiske operasjoner direkte inn i en kodeblokk som vi åpner med **Basic > Code**. Det er også mulig å skrive mer kompliserte kretser som kretser med vipper. Det kommer vi tilbake til i Øving 10. For denne øvingen holder det å vite at vi kan skrive logiske operasjoner.

To **METODER** for å skrive for eksempel kretsen i figur 9 er vist under. I første eksempel er logikken skrevet med logiske operatorer. I andre eksempel er kretsen modellert med porter.



Figur 9: Eksempelkrets

ICESTUDIO sin **Basic > Code** gir en dialog der vi må skrive inn innganger og utganger. For eksemplet i figur 9 må vi angi inngangene x , y , z og utgangene F , G .

```
krets1.v

wire w; //Vi lager en net-kobling mellom OR og AND

assign G = ~z;
assign F = w & G;
assign w = x | y;
```

KOMMANDOEN **assign** tilordner logiske operasjoner. Det vil si at i eksemplet over så vil for eksempel G endre seg hver gang z endrer seg. Rekkefølge har ingenting å si siden alt skjer samtidig i en kodeblokk i Verilog.

```
krets1.v

wire w;

and(F, w, G);
not(G, z);
or(w, x, y);
```

PORTENE **and**, **not** og **or** er primitiver i Verilog - det vil si at vi kan bruke dem direkte uten å programmere dem selv. Utgangen kommer alltid først i en primitiv, og de kan ha så mange innganger som vi måtte ønske. Rekkefølge har ingenting å si siden alt skjer samtidig i en kodeblokk i Verilog.

VERILOG-BLOKKER i Icestudio er strengt tatt ikke nødvendig, men noen kretser slik som 7-segment-dekoderen blir fort stor og uhåndterlig når den skal tegnes. Ved bruk av Verilog er det kun syv linjer kode som skal til for å

realisere en dekodeer for 7-segment-displayer.

VEKTORER I VERILOG OG ICESTUDIO

ICESTUDIO lar oss lage egne blokker. Effektiv design i Icestudio gjør bruk av kodeblokker med Verilog (Som vi lager med Basic > Code). Egne blokker er praktisk - spesielt når vi gjenbraker dem med File > Add as block.

INNNGANGER OG UTGANGER kan defineres for egne kodeblokker. Men blir det mange av disse kan også disse bli uoversiktlige. Heldigvis har Verilog en vektor-notasjon som vi kan gjenbrake i Icestudio.

VEKTORNOTASJONEN defineres med []:

Vektorer i Verilog

```

wire a0,a1,a2,a3; //lager fire nets av type wire.
reg b0,b1,b2; //Lager tre 1-bits registre
           //- det vil si tre vipper.

//Nå kan vi "assigne" verdier:
assign a0 = 1'b0;
assign a1 = 1'b1;
assign a2 = 1'b1;
assign a3 = 1'b0;
assign b2 = (a0 & a1) | (~a2 & a3);

//Med vektorer kan vi samle variabler:
wire [3:0] c;
reg [2:0] d;

//Nå kan vi skrive en og en verdi:
c[0] = 1'b0;
//eller flere verdier samtidig:
c = 4'b0101;
//Vi kan samle enkeltvariabler i vektorer:
d = {b2,b1,b0};
//Vi kan faktisk skrive til flere
//enkelvariabler samtidig:
{a3,a2,a1,a0} = 4'b0110;

```

ADRESSERINGEN i vektorer er viktig:

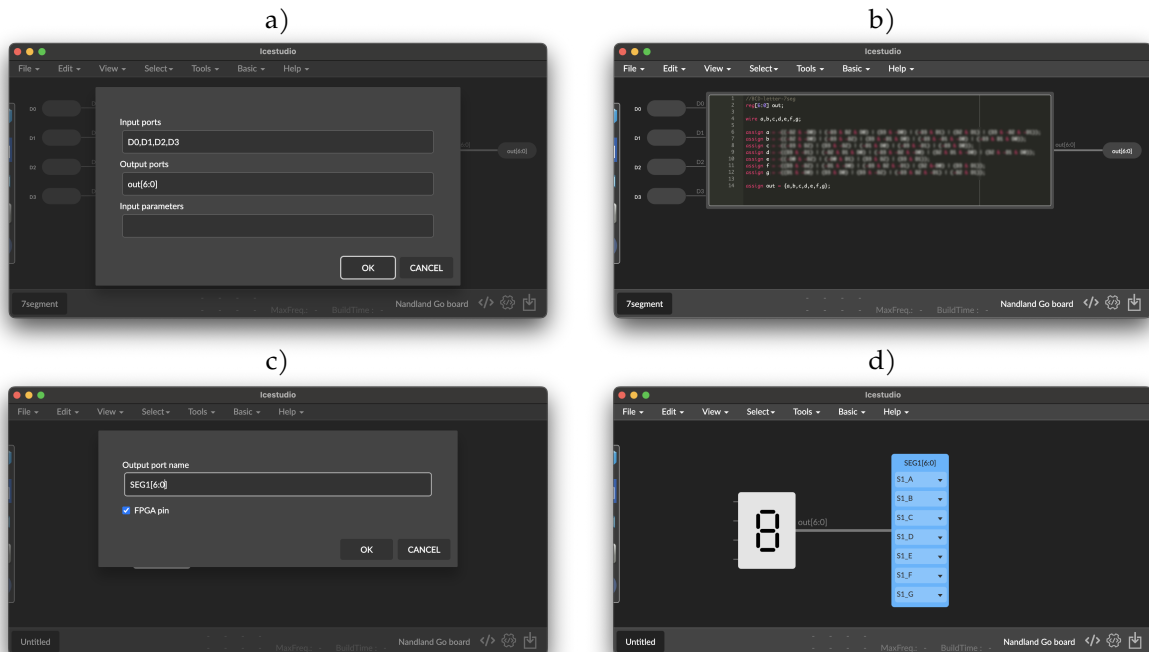
I `A[1:0]` er `A[1]` lengst til venstre slik at `A = 2'b10` betyr at vi får `A[1]` lik 1, og `A[0]` 0.

Vi kan bruke vilkårlig adressering. For eksempel er det lov å skrive `B[0:4]` og `C[14:10]`. Begge eksempler gir vektorer med fem elementer.

VEKTORER er en måte å samle data som "hører sammen".

VEKTORER I Icestudio kan spare oss for å trekke mange sammenkoblinger. Som eksempel kan vi skrive om syvsegment-dekoderen fra forrige øving. Dette er vist i figur 10:

- Dialog for å lage kodeblokk der utgangen er en vektor med syv elementer.
- Med `Basic > output` er det laget en utgang av fila ved å skrive `out[6:0]`.
- Vi kan samle utganger på kortet ved lage et Output med vektor-notasjon. Her er det brukt `SEG1[6:0]`.
- Alle syv utgangene velges i den blå boksen og den nye syvsegment-dekoderen kan kobles til syvsegment-utgangene med en (litt tykkere) strek.



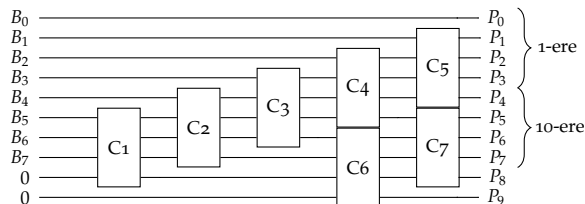
Figur 10: Bruk av vektorer i Icestudio.

BINÆR-TIL-BCD FOR SYVSEGMENT

Å bruke begge syvsegment-displayene er ikke like rett frem som å bruke ett alene. Hvordan skal for eksempel $45_{10} = 101101_2$ kunne skrives til de to syvsegmentene? Segmentet til venstre må motta $4_{10} = 0100_2$ og det til høyre 0101_2 .

C-ADD-3-ALGORITMEN løser dette problemet. Algoritmen er egentlig basert på å skifte bit til venstre og gruppere fire bit for 1-ere og fire bit for 10-ere. Hver gang en bitsekvens blir fem eller større legges tallet tre til¹.

ALGORITMEN tar utgangspunkt i at binære tall er skrevet med åtte bit. Kretsskjema som implementerer algoritmen er vist i figur 11.



DE SYV BLOKKENE I C-ADD-3 er identiske og følger sannhetstabellen i table 1.

¹ Algoritmen er beskrevet her: https://johnloomis.org/ece314/notes/devices/binary_to_BCD/bin_to_bcd.html.

Figur 11: Binær til BCD konvertering med C-Add-3.

Tabell 1: Sannhetstabell for C-add-3.

$A[3:0]$	$S[3:0]$	$A[3:0]$	$S[3:0]$
0000	0000	1000	1011
0001	0001	1001	1100
0010	0010	1010	----
0011	0011	1011	----
0100	0100	1100	----
0101	1000	1101	----
0110	1001	1110	----
0111	1010	1111	----

VIPPER

DESIGN MED D-VIPPER har to klare fordeler; D-vippene er rene minneelementer som allerede finnes i FPGA, og designen er enkel siden vippeinngangen D settes lik neste tilstand $Q(t)$.

ULEMPEN MED D-VIPPER er at kretser har tilsynelatende mye logikk for å produsere inngangsverdier for data-inngangen.

KRETSE MED T- ELLER JK-VIPPER har mindre omliggende logikk.

JK-vippe

KRETSSYMBOL FOR JK-VIPPEN er gitt i figur 12. JK-vippen har to innganger i motsetning til D-vippens ene. Karakteristisk tabell for JK-vippen er vist i table 2.

DEN KARAKTERISTISKE LIGNINGEN FOR JK-VIPPEN viser at neste tilstand er avhengig av både inngangene J og K i tillegg til utgangen (nåtilstanden) $Q(t)$:

$$Q(t+1) = J\bar{Q} + \bar{K}Q$$

JK-VIPPEN kan realiseres med en D-vippe siden

$$D = Q(t+1) = J\bar{Q} + \bar{K}Q.$$

T-vippe

KRETSSYMBOL FOR T-VIPPEN er gitt i figur 13. Karakteristisk tabell for T-vippen er vist i table 3.

DEN KARAKTERISTISKE LIGNINGEN FOR T-VIPPEN viser som karakteristisk tabell at T-vippen enten beholder utgangsverdi eller komplementerer (inverterer) utgangen:

$$Q(t+1) = T \oplus Q$$

T-VIPPEN KAN REALISERES MED EN D-VIPPE:

$$D = Q(t+1) = T \oplus Q$$



Figur 12: JK-vippe.

Tabell 2: Karakteristisk tabell for JK-vippe.

J	K	$Q(t+1)$	
0	0	$Q(t)$	Ingen endring
0	1	0	Reset
1	0	1	Sett
1	1	$\bar{Q}(t)$	Komplementering



Figur 13: T-vippe.

Tabell 3: Karakteristisk tabell for T-vippe.

T	$Q(t+1)$	
0	$Q(t)$	Ingen endring
1	$\bar{Q}(t)$	Komplementering

DESIGN MED JK-VIPPER

I DESIGN MED JK-VIPPER er det hensiktsmessig å bruke utvide den karakteristiske tabellen til JK-vippen til å inneholde nåtilstanden. Denne tabellen er laget i table 4. Radene i den høyre delen av tabellen fremkommer ved å se at hver overgang fra nå- til neste-tilstand kan lages på to måter:

- Linje 1: $Q(t) = 0$ og $Q(t+1) = 0$. Dette kan fremkomme ved "Ingen endring" og "Reset". For begge disse er $J = 1$ og derfor blir K don't care.
- Linje 2: $Q(t) = 0$ og $Q(t+1) = 1$. Dette kan fremkomme ved "Sett" og "Komplementering". For begge disse er $J = 0$ og derfor blir K don't care.
- Linje 3: $Q(t) = 1$ og $Q(t+1) = 0$. Dette kan fremkomme ved "Reset" og "Komplementering". For begge disse er $K = 1$ og derfor blir J don't care.
- Linje 4: $Q(t) = 1$ og $Q(t+1) = 1$. Dette kan fremkomme ved "Ingen endring" og "Sett". For begge disse er $K = 0$ og derfor blir J don't care.

J	K	$Q(t+1)$		$Q(t)$	$Q(t+1)$	J	K
0	0	$Q(t)$	Ingen endring	0	0	0	–
0	1	0	Reset	0	1	1	–
1	0	1	Sett	1	0	–	1
1	1	$\overline{Q}(t)$	Komplementering	1	1	–	0

Tabell 4: Utvidet Karakteristisk tabell for JK-vippe.

Eksempel

TO-BITS GRAY-TELLER realisert med JK illustrerer designmetoden.

GITT sannhetstabellen til en to-bits opp-ned Gray-teller:

x	$Q_A(t)$	$Q_B(t)$	$Q_A(t+1)$	$Q_B(t+1)$
0	0	0	0	1
0	0	1	1	1
0	1	0	0	0
0	1	1	1	0
1	0	0	1	0
1	0	1	0	0
1	1	0	1	1
1	1	1	0	1

FØRST UTVIDES sannhetstabellen med JK-inngangene til de to vippene.

Her tar vi i bruk høyre del av table 4:

x	$Q_A(t)$	$Q_B(t)$	$Q_A(t+1)$	$Q_B(t+1)$	J_A	K_A	J_B	K_B
0	0	0	0	1	0	—	1	—
0	0	1	1	1	1	—	—	0
0	1	0	0	0	—	1	0	—
0	1	1	1	0	—	0	—	1
1	0	0	1	0	1	—	0	—
1	0	1	0	0	0	—	—	1
1	1	0	1	1	—	0	1	—
1	1	1	0	1	—	1	—	0

KARNAUGH-DIAGRAMMER for inngangene gir transisjonsligninger for realisering:

$Q_A Q_B$		Q_A			
		00	01	11	10
x	0	0	1	-	-
	1	1	0	-	-

$Q_A Q_B$		Q_A			
		00	01	11	10
x	0	1	-	-	0
	1	0	-	-	1

$$J_A = \bar{x}Q_B + x\bar{Q}_B = x \oplus Q_B$$

$$J_B = \bar{x}\bar{Q}_A + xQ_A = x \odot Q_A$$

$Q_A Q_B$		Q_A			
		00	01	11	10
x	0	-	-	0	1
	1	-	-	1	0

$Q_A Q_B$		Q_A			
		00	01	11	10
x	0	-	0	1	-
	1	-	1	0	-

$$K_A = \bar{x}\bar{Q}_B + xQ_B = x \odot Q_B$$

$$K_B = \bar{x}Q_A + x\bar{Q}_A = x \oplus Q_A$$

DESIGN MED T-VIPPER

DESIGN MED T-VIPPER er enklere enn JK siden det er kun en inngang på en T-vippe.

OVERGANGEN fra nå- til neste-tilstand er vist i

Eksempel

TO-BITS GRAY-TELLER realisert med T illustrerer designmetoden.

GITT sannhetstabellen til en to-bits opp-ned Gray-teller:

x	$Q_A(t)$	$Q_B(t)$	$Q_A(t+1)$	$Q_B(t+1)$
0	0	0	0	1
0	0	1	1	1
0	1	0	0	0
0	1	1	1	0
1	0	0	1	0
1	0	1	0	0
1	1	0	1	1
1	1	1	0	1

FØRST UTVIDES sannhetstabellen med T-inngangene til de to vippene.

Her tar vi i bruk høyre del av table 5:

x	$Q_A(t)$	$Q_B(t)$	$Q_A(t+1)$	$Q_B(t+1)$	T_A	T_B
0	0	0	0	1	0	1
0	0	1	1	1	1	0
0	1	0	0	0	1	0
0	1	1	1	0	0	1
1	0	0	1	0	1	0
1	0	1	0	0	0	1
1	1	0	1	1	0	1
1	1	1	0	1	1	0

KARNAUGH-DIAGRAMMER for inngangene gir transisjonsligninger for realisering:

$Q_A Q_B$		Q_A			
x		00	01	11	10
	0	0	1	0	1
x	1	1	0	1	0
		Q_B			

$$T_A = x \oplus Q_A \oplus Q_B$$

$Q_A Q_B$		Q_A			
x		00	01	11	10
	0	1	0	1	0
x	1	0	1	0	1
		Q_B			

$$T_B = \overline{x \oplus Q_A \oplus Q_B} = \overline{T_A}$$

SKIFTREGISTER

ET SKIFTREGISTER kan implementeres med D-vipper. For et n -bits register kreves det n -vipper.

DE VANLIGSTE OPERASJONENE er skift til høyre og skift til venstre.

HØYRESKIFT = DEL PÅ 2. Dette kan illustreres med et eksempel:

$$14_{10}/2 = 7_{10}$$

$$00001110_2/2 = 00000111_2$$

Et eksempel til:

$$38_{10}/2 = 19_{10}$$

$$00100110_2/2 = 00010011_2$$

VENSTRESKIFT = MULTIPLISER MED 2. Det skiftes inn en null fra høyresiden. Dette kan illustreres med et eksempel:

$$14_{10} \cdot 2 = 28_{10}$$

$$00001110_2 \cdot 2 = 00011100_2$$

Et eksempel til:

$$38_{10} \cdot 2 = 76_{10}$$

$$00100110_2 \cdot 2 = 01001100_2$$

SKIFTREGISTRE I VERILOG

ET SKIFTREGISTER i Verilog defineres som `reg [3:0] mittregister;`

ET HØYRESKIFT gjøres med operatoren `>>` 1. Det vil da legges inn en null på MSB og alle bit flyttes et hakk til høyre. LSB mistes i prosessen.

ET VENSTRESKIFT gjøres med operatoren `<<` 1. Det vil da legges inn en null på LSB og alle bit flyttes et hakk til venstre. LSB mistes i prosessen.

SKIFTREGISTEROPERASJONER BØR KLOKKES:

Skiftregistre i Verilog

```

module skiftregister (mittregister, inndata, clk)
    //Mottar et 4-bits register
    //og initialiserer det til 14
    output reg [3:0] mittregister = 14;
    input clk;
    input inndata; //et databit

    //En always for høyreskift
    always @(posedge clk)
    begin
        mittregister <= mittregister >> 1;
        mittregister[3] <= inndata;
    end

    //En always for venstreskift
    always @(posedge clk)
    begin
        mittregister <= mittregister << 1;
        mittregister[0] <= inndata;
    end
endmodule

```

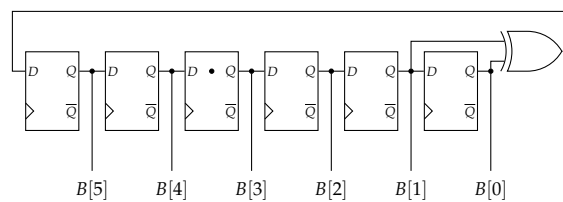
GENERATOR FOR TILFELDIGE TALL

EN TILFELDIG sekvens av informasjon kan være viktig for å teste ut systemer eller brukes direkte i for eksempel koding av informasjon.

EN KOMPLETT TILFELDIG sekvens er slik at alle tall mellom minste og laveste verdi opptrer like ofte.

EN GOD MÅTE å lage en komplett tilfeldig sekvens for logiske kretser er å bruke et skiftregister og en XOR-port. For et n -bits skiftregister vil alle tall mellom 1 og 2^n opptre i en sekvens. Sekvensen repeteres riktignok ved gjentatte kjøring, men gjøres den lang nok vil den fremstå som tilfeldig.

KRETSEN i figur 14 er en slik komplett generator (engelsk: Random Number Generator (RNG)).



Figur 14: 6-bits slumptallsgenerator.